

Spiking neural network solvers for model predictive control of autoclave composite curing

Eisa Shaiju

Abstract

Model predictive control (MPC) requires solving a constrained quadratic program (QP) at every control interval, which places a hard computational floor on the achievable sampling rate and motivates interest in solvers that map onto parallel, event-driven hardware. Recent work has shown that the time-averaged firing rates of certain inhibition-dominated spiking neural networks (SNNs) yield the solutions of linear and quadratic programs, suggesting that an SNN could serve as a drop-in QP solver inside an MPC loop. Here we test that proposition on a demanding physical problem: the autoclave cure of a thick-sectioned composite laminate, whose exothermic Arrhenius kinetics make the linearised prediction model unstable precisely where control is most critical. We implement a non-linear finite-difference digital twin, a classical CVXPY/OSQP MPC baseline, and an SNN-QP controller built on a projected-gradient/LIF solver, and we subject the claim of solver equivalence to direct measurement. We report three findings. First, establishing that two controllers solve the *same* QP is substantially harder than it appears: we identify and correct a one-step prediction-window error and an asymmetric Jacobian clamp, either of which silently invalidates a head-to-head comparison, and we unify both controllers onto a single canonical QP construction verified bit-identical across a full trajectory. Second, the per-step QP is *infeasible* at stiff exotherm steps under the natural hard-constraint formulation — a reference interior-point solver agrees on all twelve configurations tested — so the SNN's apparent failure to converge was, for a large part of the trajectory, the correct response to an empty feasible set. Third, once the problem is made feasible, the SNN reaches the optimum to within 5×10^{-4} °C of a reference solver, yet its convergence flag never fires, because the library's projected-gradient stopping test is absolute rather than scale-invariant and cannot trigger on a problem whose gradient scale is $\sim 10^{10}$. We conclude that the two controllers solve the same QP and that their applied controls agree to 0.714 °C RMS, but that reliable convergence is not established, and we identify the scale-invariant stopping test as the specific remaining obstacle.

1 Introduction

Thick-sectioned polymer-matrix composites are cured in an autoclave under a prescribed temperature profile. The governing difficulty is that the resin's polymerisation is exothermic: past the gelation point the reaction liberates heat faster than the low-conductivity laminate can shed it, so the interior of the part can run away thermally even while its surface is being cooled. Standard practice uses open-loop ramp-and-hold recipes, which cannot react to this, and which consequently trade cycle time and part quality for safety margin.

Model predictive control is the natural remedy, since it can anticipate the exotherm over a prediction horizon and brake before the interior overshoots. Its cost is computational: MPC requires a constrained QP solution every sampling interval. This has motivated a broad search for QP solvers suited to specialised hardware, and in particular for solvers whose structure maps onto parallel or event-driven substrates rather than sequential CPUs.

A line of theoretical work has established that spiking neural networks are intimately connected to convex optimisation: the connectivity, thresholds, and time constants of a network of leaky integrate-and-fire (LIF) neurons can be placed in correspondence with the parameters of a linear or quadratic program, such that the network's output solves that program [2]. If this correspondence holds in practice, an SNN is not merely a biologically-inspired analogue of a solver — it *is* a solver, one that is natively event-driven and therefore a candidate for low-power neuromorphic or FPGA deployment.

This report asks whether that candidacy survives contact with a hard control problem. The question is not whether an SNN can solve *some* QP; it is whether an SNN can be substituted for a classical solver inside a closed control loop, on a plant stiff enough to be interesting, and produce demonstrably the same control. We take the phrase "demonstrably the same" seriously, and much of what follows concerns the surprising difficulty of establishing it.

Our contributions are:

1. A complete simulation environment — non-linear plant twin, classical MPC baseline, and SNN-QP controller — in which both controllers are driven from a single shared QP construction, so that solver equivalence is testable rather than assumed (Section 2, Section 3.1).
2. The identification of two distinct failure modes that each invalidate a naive comparison and which are individually invisible in aggregate closed-loop metrics: a one-step prediction-window offset, and an asymmetric regularisation of the Arrhenius Jacobian (Section 3.2).
3. The finding that the per-step QP is genuinely infeasible through the exotherm under hard state constraints, which reframes the solver's behaviour and which no amount of preconditioning could have repaired (Section 3.3).
4. A precise diagnosis of the residual non-convergence as an artefact of an absolute, rather than scale-invariant, stopping criterion, together with evidence that the returned solution is nonetheless correct (Section 3.4).

2 Methods

2.1 Plant model

The laminate and its tooling are discretised as a one-dimensional conduction problem with three composite nodes and four tooling nodes. The state is

$$\mathbf{x} = [T_{c,1}, T_{c,2}, T_{c,3}, T_{t,1..4}, \alpha_1, \alpha_2, \alpha_3]^\top \in \mathbb{R}^{10},$$

where T_c are composite temperatures, T_t tooling temperatures, and α the local degree of cure. The single actuator is the autoclave air temperature T_a , which enters at the outer tooling node.

Cure advances by the single-term autocatalytic Arrhenius law

$$\frac{d\alpha}{dt} = A_c \exp\left(-\frac{E_a}{RT}\right) \alpha^m (1 - \alpha)^n, \quad (1)$$

and liberates heat into the composite energy balance in proportion to da/dt . The plant is integrated by explicit finite differences at $T_E = 60$ s and is never linearised: it is the ground truth against which both controllers are judged. All physical parameters are taken from the reference formulation [1] and were verified term-by-term against its appendix.

Open-loop simulation reproduces the phenomenon of interest: under a static ramp-and-hold profile the part centre exceeds 140 °C, well above the surface temperature and into thermal degradation.

2.2 Model predictive control formulation

At each interval both controllers linearise the plant about the current operating point $(T, \bar{a})$, obtaining a discrete-time model (A_p, B_p) . Differentiating (1) produces the Jacobian terms

$$\frac{\partial \dot{a}}{\partial T} = f_0 \frac{E_a}{RT^2}, \quad \frac{\partial \dot{a}}{\partial a} = f_0 \left(\frac{m}{a} - \frac{n}{1-a} \right), \quad (2)$$

with f_0 the reaction rate at the operating point. The second expression has genuine poles at $a \rightarrow 0$ and $a \rightarrow 1$; we therefore evaluate it at a clamped to $[10^{-3}, 0.999]$. This regularises the *point at which the derivative is evaluated*, not the model, and is applied identically to both controllers.

The MPC problem over a horizon N penalises tracking error, control effort, and control slew, subject to actuator limits and a spatial-uniformity constraint:

$$\min_u \sum_{k=0}^{N-1} \|x_k - x_{\text{ref}}\|_Q^2 + Ru_k^2 + S(u_k - u_{k-1})^2 \quad (3)$$

$$\text{s. t. } x_{k+1} = A_p x_k + B_p u_k, \quad T_{a, \min} \leq u_k \leq T_{a, \max}, \quad |u_k - u_{k-1}| \leq \Delta_{\max}, \quad |x_k^{(1)} - x_k^{(3)}| \leq G_{\max}.$$

The final constraint bounds the centre-to-surface temperature difference and is what enforces a uniform, inside-out cure.

2.3 Canonical condensed form

Eliminating the state recursion yields a dense QP in the input sequence $z = [u_0, \dots, u_{N-1}]^T$:

$$\min_z \frac{1}{2} z^T H z + f^T z \quad \text{s. t. } A_{\text{ineq}} z \leq b_{\text{ineq}}, \quad (4)$$

with $H = 2(\Gamma^T Q \Gamma + R + D^T S D)$ and $f = 2\Gamma^T Q(\Phi x_0 - x_{\text{ref}}) - 2D^T S d_0$, where Φ and Γ are the free- and forced-response matrices and D is the first-difference operator. Row i of Φ is A_p^i , predicting x_i ; this indexing is load-bearing and is revisited in Section 3.2.

Both controllers call one implementation of (4). The CVXPY/OSQP baseline solves it directly. The SNN controller applies a Jacobi change of variables before solving, described next.

2.4 SNN-QP solver and preconditioning

The SNN solver minimises $1/2 x^T A x + b^T x$ subject to $Cx + d \leq 0$ by alternating a gradient step with a greedy projection onto violated constraint boundaries — the LIF analogue being that a neuron's membrane potential is integrated until it crosses threshold, at which point a spike applies a discrete corrective displacement [2]. Constraints map to thresholds; the constraint Gram matrix CC^T plays the role of the recurrent weight matrix.

Because H in (4) is stiff, the SNN adapter applies Jacobi scaling with $D_s = \text{diag}(\sqrt{\text{diag } H})$:

$$\hat{H} = D_s^{-1} H D_s^{-1}, \quad \hat{g} = D_s^{-1} f, \quad z = D_s^{-1} \hat{z}, \quad (5)$$

with constraint rows subsequently normalised. This is an exact change of variables; the solution is mapped back by $z^* = \hat{z}^* / D_s$ before use, and all reported residuals and objectives are evaluated on the *original* problem (4), never on the scaled surrogate. We note that the reference solver applies its own internal equilibration, so (5) is not an asymmetry between the two controllers so much as an explicit statement of what the reference does implicitly.

2.5 Evaluation protocol

Head-to-head numbers are generated by a single harness driving two independent plant instances from identical initial conditions, with disturbances injected at the same point in the per-step sequence for both branches. This matters: two separately-written simulation scripts had injected the disturbance on opposite sides of the control computation, producing a one-step misalignment that quietly invalidated any step-wise comparison between them.

We distinguish four questions that are frequently conflated:

- **Feasibility** — does the returned point satisfy $A_{ineq} z \leq b_{ineq}$ to tolerance, measured on the original problem?
- **Optimality** — is its objective close to a reference solver's on the identical arrays?
- **Applied-move agreement** — does u_0 , the only quantity reaching the plant, match?
- **Formal convergence** — did the solver's own stopping criterion fire, and if not, why?

Only the third determines closed-loop behaviour; the others explain it. We report a lower objective attained at an infeasible point as a failure, never as a success.

3 Results

3.1 A first implementation, and two bugs that flattered it

An initial SNN-QP controller closed the loop and produced a superficially reasonable trajectory, but was worse than the baseline on every metric and roughly a hundred times slower (≈ 13 s per solve). Two defects were responsible.

The first was mundane: the solver was running its pure-Python reference path rather than its compiled kernel. Switching backends reduced a single solve from ≈ 5300 ms to ≈ 60 ms with bit-identical output.

The second was substantive. The condensed builder contained a stabilisation step that rescaled the prediction matrix whenever it became unstable,

```
rho = max(abs(eigvals(Ap))); if rho >= 1: Ap *= 0.98 / rho
```

The intent was conditioning; the effect was to delete the exotherm from the prediction. During gelation $\rho(A_p)$ genuinely exceeds 1 — the linearised model is *correctly* predicting thermal runaway, and that prediction is exactly the signal the controller needs in order to brake in time. With it suppressed, the controller held the autoclave at maximum temperature through the exotherm and overshoot by 20 °C against the baseline's 13 °C. Removing the rescaling restored correct braking.

The lesson generalises beyond this codebase: an unstable prediction model is not automatically a numerical problem to be suppressed. Here instability was the physics.

3.2 Establishing that two controllers solve the same problem

With those fixes the two controllers produced similar aggregate metrics, and it was tempting to declare equivalence. Direct measurement did not support it, for two independent reasons.

A one-step prediction window offset. The condensed builder constructed Φ with exponent $i+1$ rather than i , so its cost and constraints applied to states $x_1 \dots x_N$ while the baseline's applied to $x_0 \dots x_{N-1}$. Both windows yield well-posed QPs; both produce plausible trajectories. Reconstructing each window and solving both with the *same* reference solver, so that only the window varied, showed agreement with the live baseline to 0.018 °C for the correct window and disagreement of 1.20 °C for the shipped one — with the discrepancy concentrated, predictably, where the dynamics are least linear.

An asymmetric Jacobian clamp. The SNN controller bounded the exotherm Jacobian terms of (2) while the baseline did not. At a benign operating point this is negligible ($\max |A_p| \approx 10^{-4}$); at a gelation-like point it reaches ≈ 1851 . The two controllers were therefore predicting with materially different models exactly where the comparison mattered.

Both were removed by routing both controllers through one canonical construction (Section 2.3) and by making the clamp an explicit parameter defaulting to *off* on both. Verification: building both controllers' QPs from the same state at each of 160 trajectory steps, including the disturbance and the full exotherm, gives a maximum difference of exactly zero across H, f, A_{ineq}, b_{ineq} , the bounds, and the variable ordering.

We stress how weak aggregate metrics are as evidence here. Before the fixes, the two controllers matched to within a few degrees on peak overshoot while diverging by as much as 64 °C in applied control mid-trajectory. Summary statistics can agree while the underlying trajectories do not.

3.3 The stiff QP is infeasible, not merely ill-conditioned

With the comparison made fair, the SNN still exhausted its iteration budget at every step and returned points violating the constraints by large margins. The natural hypothesis, and the one we initially pursued, was conditioning: the condensed Hessian is assembled from weights spanning three orders of magnitude and compounded through I , whose entries involve A_p^k .

That hypothesis is false, and we can exclude it directly. Replacing the Jacobi scaling (5) with exact eigenwhitening, so that $\text{cond}(\text{hat } H) = 1.0$ — the best-conditioned Hessian obtainable — changed the final feasibility by less than 3 %. A perfectly conditioned Hessian did not help.

Sweeping 24 configurations across clamp setting, constraint form, horizon, and step size located the real cause. **All twelve hard-constrained configurations are reported infeasible by the reference solver**, at every horizon and under both prediction models. The mechanism is visible in the constraint offsets: the uniformity constraint at horizon step $N-1$ carries an offset $\propto G A_p^{N-1} x_0$, amplified by $\rho(A_p)^{N-1} \approx 4200$ at the gelation peak. The feasible set is empty. The cold-start point is violated by 2.5×10^5 before a single iteration executes.

This reframes the entire diagnosis. The solver was not failing to solve a hard problem; it was being asked to locate a point inside a region that does not exist, and its refusal to converge was in that sense correct. It also explains the whitening result: the offending term is independent of the decision variable, so no transformation $z = D \text{hat } z + z_0$ can shrink it.

Introducing slack variables on the predicted-state rows only — actuator limits are real physical bounds and remain hard — with an exact penalty restores feasibility. The linear penalty term drives slacks to zero wherever the hard constraint is attainable (observed magnitude 10^{-29} at a benign state), so nothing is relaxed that could have been satisfied. Applied identically to both controllers, this preserves the parity

established in Section 3.2.

3.4 Correct answers under a mis-specified stopping test

With a feasible problem the SNN returns feasible points, and the closed-loop numbers improve substantially (Table 1). The formal convergence flag, however, still never fires.

The blocker is identifiable. The projected-gradient norm at the returned point is $\approx 1.66 \times 10^{10}$ and is unchanged by a six-fold increase in iteration budget. The solver tests this norm as an *absolute* quantity against a tolerance of 5×10^{-2} . On a problem whose gradient scale is 10^{10} , satisfying that test demands twelve orders of magnitude of reduction; the criterion is not scale-invariant and cannot fire regardless of solution quality.

To determine whether the returned point is nonetheless correct, we evaluated at a horizon short enough that the A_p^{N-1} amplification is mild, and compared against a reference solve of the identical problem:

Quantity	Value
Applied move u_0 vs. reference optimum	agree to 5×10^{-4} °C
Relative objective gap	-5.4×10^{-8}
Constraint residual	5.4×10^{-4} (feasible)
Projected-gradient norm, <i>relative</i> to gradient scale	9.7×10^{-3} — inside the 5×10^{-2} tolerance
Reported converged flag	False

On a well-posed stiff QP the SNN reaches the optimum, and a scale-invariant form of the solver's own criterion would have fired. The persistent negative flag is a property of the stopping test, not evidence of a wrong answer. This is, we suggest, the most transferable finding here: absolute tolerances on gradient norms are unsafe for condensed MPC problems, whose objective scale varies by orders of magnitude with the operating point.

3.5 Closed-loop results, and a degenerate configuration

Table 1 gives the controlled comparison at the settled configuration.

Table 1. Controlled head-to-head, horizon 10, soft state constraints, identical prediction model. Trajectory difference is the Euclidean norm over composite temperatures and cure states.

Metric	Nominal	Disturbance	Exotherm window
RMS applied-control difference	0.714 °C	0.791 °C	0.559 °C
RMS trajectory difference	0.249	0.279	0.414
Max applied-control difference	3.57 °C	3.95 °C	0.65 °C
SNN max constraint residual	1.55	2.07	1.55
SNN formal convergence rate	0 %	0 %	0 %
Steps feasible enough to score objective gap	48.1 %	58.8 %	45.2 %
Mean objective gap on feasible steps	1.05×10^{-4}	1.22×10^{-4}	5.02×10^{-5}

Applied moves corrected by safety filter	13.1 %	15.0 %	29.0 %
--	--------	--------	--------

Relative to the pre-correction configuration, RMS applied-control difference falls from 16.005 °C to 0.714 °C and the maximum constraint residual from 1.85×10^5 to 1.55. Both controllers achieve full uniform cure with comparable overshoot (13.77 °C baseline, 13.24 °C SNN) and identical actuator-limit compliance.

A cautionary configuration. At horizon 5 the comparison appears flawless: RMS applied-control difference 0.000 °C, formal convergence 98.8 %, zero clipping, zero constraint violations. It is also useless. A five-minute horizon cannot see past the thermal transport lag, so the effort term dominates, the controller drives the actuator to its lower bound, the part cools from 28 °C to 11 °C, and the degree of cure never departs from zero. Both controllers agree perfectly because both are inert, saturated against the same bound. We report this because it is the precise failure mode that an equivalence study is most likely to mistake for success, and because it is invisible in every metric except the cure state itself. The same caution applies more weakly during heat-up, where 41.7 % of steps have both controllers pinned at the slew limit; agreement on those steps reflects a shared actuator bound and is excluded from our claims.

4 Discussion

We set out to test whether a spiking network can be substituted for a classical QP solver inside an MPC loop. Our answer is qualified in a specific way: the two controllers demonstrably receive the same problem, and their applied controls agree to 0.714 °C RMS with an objective gap of order 10^{-4} where it can be measured, but formal convergence is attained on no step at any horizon that actually cures the part, and roughly 13 % of applied moves are corrected by a downstream safety filter. We therefore state the result as *same problem, unreliable convergence*, and decline to call it equivalence.

Three observations seem worth carrying forward.

Equivalence is an experimental claim with a high evidentiary bar. Two of the defects we found — the prediction-window offset and the asymmetric Jacobian clamp — produced closed-loop trajectories that looked entirely reasonable and aggregate metrics that were close. Neither is detectable without constructing both QPs and comparing them numerically. We would suggest that any claim of solver substitutability be accompanied by a per-step array-level comparison, not a metrics table.

Infeasibility masquerades as non-convergence. For a substantial portion of the trajectory the problem we posed had no solution, and every diagnostic we ran on the solver was, in retrospect, measuring the consequences of that. The signature — iteration budget exhausted, large residuals, lower-than-optimal objective at an infeasible point — is indistinguishable from a struggling solver until a reference solver is asked the same question. Condensed MPC formulations with state constraints and unstable prediction models appear especially prone to this, since the constraint offsets inherit $\rho(A_p)^{N-1}$ amplification while the constraint gradients do not.

Stopping criteria must be scale-invariant. The solver studied here reaches the optimum and reports failure, because its convergence test compares an absolute gradient norm against a fixed tolerance on a problem whose natural scale is 10^{10} . This is a small implementation detail with a large consequence, and it will recur in any application where the objective scale varies with the operating point — which is to say, in most non-linear MPC.

Limitations. The horizon reduction that made the problem tractable was selected empirically rather than

from the plant's thermal time constants. Softening the uniformity constraint weakens its guarantee, though the hard form it replaces was infeasible and therefore unimplementable. Our evidence that the returned solution is optimal comes from a horizon too short to control the plant; at working horizons we establish feasibility but not optimality. Roughly half of steps at the working horizon remain outside the tolerance at which an objective gap can be evaluated. The reference solver itself reports infeasibility on 13 % of steps under the hard formulation, a symmetric anomaly we have not investigated.

Future work. The immediate next step is narrow and well-defined: wrap a scale-invariant convergence test around the solver and re-measure the convergence rate at the working horizon. This is the single item standing between the present result and a clean convergence number, and it should precede any hardware implementation. Committing a solver to fixed hardware while it exhausts its iteration budget every step and depends on a downstream filter for 13 % of its outputs would reproduce the present difficulty in a substrate where it is far more expensive to diagnose. The architectural argument for event-driven hardware is unaffected by this and remains the motivation for the work; it is simply not yet supported by a convergence result.

Broader impact

The nearer-term impact of work in this direction is on process efficiency: closed-loop cure control shortens cycle times and reduces scrap in composite manufacturing, with corresponding reductions in energy and material waste relative to conservative open-loop recipes. A longer-term impact concerns where such controllers can run — solvers that map onto low-power event-driven hardware make predictive control viable on embedded and edge platforms that cannot support a conventional optimisation stack.

Two cautions attach. First, a controller that appears to track a reference correctly while its underlying solver has not converged is a genuine safety concern in any process where constraint satisfaction protects the product or the equipment, and the present work is largely an illustration of how easily that state can be mistaken for success. We would argue that convergence and feasibility should be logged and monitored in deployment, not merely checked in simulation. Second, the same argument that makes neuromorphic solvers attractive for industrial edge control makes them attractive for autonomous systems generally, including applications whose desirability is less clear; that trade-off is not specific to this work but is not absent from it either.

References

- [1] P. Dufour, D. J. Michaud, Y. Touré, and P. S. Dhurjati. A partial differential equation model predictive control strategy: application to autoclave composite processing. *Computers & Chemical Engineering*, 28(4):545–556, 2004.
- [2] A. Mancoo, S. W. Keemink, and C. K. Machens. Understanding spiking networks through convex optimization. *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
- [3] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd. OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12:637–672, 2020.
- [4] S. Diamond and S. Boyd. CVXPY: a Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.